

L1 —什么是 Agent? (Layman → Beginner 起步)

目标：讲完这节，你能一句话说清「agent 和普通调 LLM 的区别」，并看懂一个 agent loop 的骨架。这是整门课的地基，概念对了，后面写代码才不会晕。

1. 一个类比：点菜 vs. 请厨师

- **普通调 LLM**：你给模型一段 prompt，它返回一段文字。就像你把菜谱念给一个人听，他复述一遍。**一问一答，模型不「动手」，也不「循环」。**
- **Agent**：你给它一个**目标**（「帮我订一张明天去上海的高铁票」），它自己决定 **要做哪些步骤、调用哪些工具（查车次、比价、下单）、看到结果后再决定下一步**，直到目标达成。就像你请了个厨师，说「做顿川菜」，他自己买菜、切配、掌勺。

一句话：Agent = LLM当大脑 + 工具当手脚 + 记忆 + 一个「想 → 做 → 看结果 → 再想」的循环。

2. Agent 的四个核心部件

部件	作用	客服场景的例子
LLM (大脑)	理解、推理、决定下一步做什么	理解用户在问「退货政策」
Tools (手脚)	让模型能「做事」，突破纯文本	查订单数据库、查物流、发起退款
Memory (记忆)	记住对话历史和已知事实	记得用户上一句说的订单号
Loop (循环)	反复「决策 → 行动 → 观察」直到完成	查完订单发现要查物流，再查一次

关键洞察：LLM 本身不会「做事」，它只会**输出文字**。所谓「调用工具」，本质是 模型输出一段结构化文字（比如 {"tool": "查订单", "args": {"id": "123"}}），**你的代码**读到它、真的去执行、把结果再喂回给模型。模型和真实世界之间，永远隔着 你写的一层代码。这一点想通了，agent 就不神秘了。

3. Agent Loop 的骨架 (伪代码)

这是整门课最重要的一张图，后面所有内容都是往这个骨架上加东西：

```
目标 = "帮用户查订单 123 的物流"
messages = [系统提示, 用户消息]
```

循环：

```
    回复 = LLM(messages, 可用工具列表)           # 想：模型决定下一步

    如果 回复里要求调用工具：
        结果 = 执行工具(回复.工具名, 回复.参数)   # 做：你的代码真去执行
        messages.append(回复)                     # 记：把模型的决定记下来
        messages.append(结果)                     # 记：把工具结果记下来
        继续循环                                   # 看结果 → 回到「想」
    否则：
        返回 回复.文本                             # 模型觉得够了，给最终答案
```

跳出循环

盯住三件事：1. 是个 `while` 循环，不是一次调用——这是 agent 的灵魂。2. 工具由模型「请求」、由你的代码「执行」——二者分离。3. 每一步的决定和结果都追加进 `messages` ——这就是「上下文」在增长，也埋下了你关心的上下文长度问题（第 6 课专门讲）。

4. 什么时候「不该」用 agent? (求职加分点)

新手容易过度设计。记住：能用一次 LLM 调用解决的，就别搭 agent。

- 固定流程（翻译、摘要、分类）→直接调 LLM，或用简单的 prompt chaining，别上 loop。
 - 只有当任务步骤数不确定、需要根据中间结果动态决策、要与外部系统交互时，agent 才划算。
 - 面试时你能说出「我评估过这个场景其实不需要 agent，用 XX 更简单」，比硬套 agent 更能体现工程判断力。
-

5. 本节小结

- Agent = LLM + 工具 + 记忆 + 循环，核心是那个「想 → 做 → 看 → 再想」的 loop。
- LLM 只输出文字；真正执行工具的是你的代码，模型与世界之间隔着一层。
- 上下文 (`messages`) 会随循环不断增长——记住这个伏笔。
- 不是所有任务都值得上 agent，能一次调用搞定就别绕。

下节课 (L2) 我们把这个伪代码变成能跑的 Python，你会亲手写出第一个 agent loop。